

Bas — CM 2

Éléments de shell de Bourne (métacaractères & redirections) · UE Bases de l'Architecture et des Systèmes · L1 Informatique, UPS · Semestre 1 2026-2027

1. Métacaractères du shell

Les **métacaractères** sont des caractères génériques (« jokers ») qui permettent de **construire des listes de noms de fichiers** et de **modifier l'interprétation des commandes** par le shell. Exemples de métacaractères : `*`, `?`, `\.`

Construire des listes de noms de fichiers

Problème typique : dans un répertoire, effacer les fichiers d'extension « jpg » sans toucher aux autres, ou effacer `image001.jpg...image999.jpg`. **Solution** : construire la liste des fichiers correspondant à un modèle (motif). Les métacaractères servent à construire ces **listes de listes de noms de fichiers**, séparés par des espaces, correspondant à un modèle donné. Si aucun nom de fichier ne correspond au modèle, **le modèle n'est pas remplacé** (il reste tel quel).

Motif	Signification
<code>*</code>	une chaîne de caractères quelconque (éventuellement vide) ne contenant pas de caractère <code>/</code> , et ne commençant pas par un caractère <code>.</code> si <code>*</code> est placé en début de modèle.
<code>?</code>	un caractère quelconque, à l'exception d'un caractère <code>/</code> , ou d'un caractère <code>.</code> si <code>?</code> est en début de modèle.
<code>[liste]</code>	un caractère dans la liste placée entre crochets : par énumération, par intervalle (avec <code>-</code>), ou par négation (avec <code>!</code> juste après le crochet ouvrant).

- `[Aa]` désigne le caractère A ou le caractère a
- `[0-9a-zA-Z]` désigne un caractère alphanumérique quelconque
- `[!0-9]` désigne n'importe quel caractère sauf un chiffre

Le shell commence par interpréter ces métacaractères : il forme les listes des noms de fichiers qui correspondent au modèle, puis **transmet ces noms de fichiers aux commandes** qui sont alors exécutées. Les commandes reçoivent en paramètre le résultat de cette interprétation.

```
rm *.jpg
rm image???.jpg
```

Modifier l'interprétation des commandes

Les autres métacaractères changent la façon dont le shell découpe et interprète la ligne de commande elle-même (séparation, citations, redirections, variables...) :

Car.	Rôle
<code>;</code>	sépare plusieurs commandes situées sur une même ligne.
<code>' '</code>	délimite une chaîne contenant des espaces ; à l'intérieur, tous les métacaractères perdent leur signification.
<code>" "</code>	délimite une chaîne contenant des espaces ; à l'intérieur, tous les métacaractères perdent leur signification, sauf <code>`</code> , <code>\$</code> et <code>\</code> .
<code>` `</code>	« capture » la sortie standard d'une commande pour former un nouveau paramètre ou une nouvelle commande.
<code><</code>	réalise une redirection de l'entrée standard des commandes.

>	réalise une redirection de la sortie standard des commandes.
&	suivi d'un chiffre, désigne les unités standard lors des redirections. Placé après une commande, lance cette commande en arrière-plan.
	réalise un branchement de commandes (« pipe »).
\$	retourne la valeur de la variable qui suit (pas d'espace entre \$ et le nom de la variable) ; joue aussi un rôle vis-à-vis des paramètres d'un shell.
\	protège le caractère qui suit (normal ou métacaractère), sauf à l'intérieur d'une chaîne délimitée par '. À l'intérieur d'une chaîne délimitée par ", il n'est interprété comme métacaractère que s'il précède l'un des 4 caractères ", ` , \$ ou \ ; dans tous les autres cas il n'est pas interprété.

Exemple : la chaîne "*" est interprétée de la même façon que la chaîne '*' (le \ protège le * dans les deux cas).

2. Redirections

Unités standard

Chaque processus a accès à trois **unités logiques** : l'entrée standard (`stdin`, par défaut le clavier), la sortie standard (`stdout`, par défaut l'écran) et la sortie standard des erreurs (`stderr`, par défaut l'écran). Les **redirections** permettent de changer les connexions par défaut entre ces unités logiques et les unités physiques : `stdin` ← clavier, `stdout` → écran, `stderr` → écran.

Filtres

Certaines commandes peuvent recevoir des données par l'entrée standard (`stdin`) et fournir des informations sur la sortie standard (`stdout`, résultats) et la sortie standard des erreurs (`stderr`, messages d'erreur). Ces commandes sont des « **filtres** ». Les filtres peuvent être « **branchés** » les uns à la suite des autres : le `stdout` du premier devient le `stdin` du suivant.

- clavier → `stdin` → **Filtre1** → `stdout` → `stdin` → **Filtre2** → `stdout` → écran
- une commande sans résultat sur `stdout` mais recevant des données sur `stdin` peut se trouver en **fin** de branchement.
- une commande sans données reçues sur `stdin` mais fournissant un résultat sur `stdout` peut se trouver en **début** de branchement.

Quelques commandes

Commande	Rôle
<code>echo [chaîne...]</code>	affiche (sur <code>stdout</code>) les chaînes de caractères passées en paramètres.
<code>cat [fichier...]</code>	affiche le contenu des fichiers dont les désignations sont en paramètres. Sans paramètre, affiche le contenu de <code>stdin</code> .
<code>grep chaîne [fichier...]</code>	affiche les lignes des fichiers en paramètres (ou de <code>stdin</code> si aucun paramètre) qui contiennent au moins une occurrence de <code>chaîne</code> .
<code>wc -l [fichier...]</code>	affiche le nombre de lignes, suivi des désignations des fichiers passés en paramètre (ou le nombre de lignes de <code>stdin</code> si aucun paramètre).

Redirection de l'entrée standard — entrée par défaut (au clavier)

Quand une commande attend des lignes sur `stdin`, on peut les taper au clavier avec l'une de ces deux syntaxes :

Syntaxe 1 (interactive)	Syntaxe 2 (utilisable en script)
-------------------------	----------------------------------

Commande Ligne1 Ligne2 ... <Ctrl>D	Commande << séparateur Ligne1 Ligne2 ... séparateur
--	---

Exemple (en bleu, ce qui est tapé par l'utilisateur) :

<pre>\$ cat bonjour bonjour monsieur monsieur <Ctrl>D \$</pre>	<pre>\$ cat << FIN >bonjour >monsieur FIN bonjour monsieur \$</pre>
--	---

- **Syntaxe 1** : <Ctrl>D (caractère non éditable, code ASCII 4) simule la fin d'un fichier en mode interactif. Le texte tapé est envoyé à la commande **ligne par ligne** — syntaxe inadaptée pour certaines commandes (ex. une commande qui devrait retourner les deux dernières lignes de l'entrée standard, puisqu'elle ne les connaît pas encore quand elle les traite).
- **Syntaxe 2** : les deux caractères << doivent être collés. Cette syntaxe peut être utilisée dans les scripts (contrairement à la première, qui suppose un utilisateur au clavier).

commande < désignation_fichier : l'entrée standard de commande est connectée au fichier, ouvert en mode **lecture**.

```
grep 'e' < fich_entree.txt
```

affiche les lignes du fichier `fich_entree.txt` qui contiennent au moins un e.

Redirection de la sortie standard d'une commande

- commande > désignation_fichier (**redirection simple**) : la sortie standard de commande est connectée au fichier. S'il n'existe pas, il est créé ; sinon son contenu est **écrasé**. Le fichier est ouvert en mode « écriture ».
- commande >> désignation_fichier (**avec concaténation**) : si le fichier n'existe pas, il est créé ; sinon la sortie est **ajoutée en fin** de fichier. Le fichier est ouvert en mode « écriture en fin de fichier ».

```
grep 'e' > fich_sortie.txt # ecrase / cree fich_sortie.txt
grep 'e' >> fich_sortie.txt # ajoute a la fin de fich_sortie.txt
```

Dans les deux cas : écrit dans `fich_sortie.txt` les lignes tapées au clavier qui contiennent au moins un e (l'entrée reste le clavier, seule la sortie change).

commande >&2 : la sortie standard de commande est connectée à sa **sortie standard des erreurs**. Très utile pour afficher des messages d'erreurs dans les scripts. Les caractères >&2 doivent être **collés**.

```
echo "Fichier introuvable !" >&2
```

affiche la chaîne `Fichier introuvable !` sur `stderr`.

Branchement de commandes (pipe)

commande1 | commande2 : le métacaractère | est appelé « pipe » (ou « tube »). La sortie standard de commande1 est connectée à l'entrée standard de commande2. Ce branchement permet de réaliser des **enchaînements de commandes**, sans passer par un fichier intermédiaire.

```
cat fich.txt | grep 'e' | grep '2' | wc -l
```

affiche à l'écran le nombre de lignes du fichier `fich.txt` qui contiennent à la fois au moins un caractère e et au moins un caractère 2 (chaque `grep` filtre à son tour la sortie du précédent).

Capture de la sortie standard d'une commande

``commande`` (ou `$(commande)`) : la sortie standard de `commande` est transformée en une **chaîne de caractères** dans laquelle les éventuels sauts de ligne sont transformés en espaces. Cette chaîne peut constituer une autre commande, ou être utilisée pour former les paramètres d'une autre commande.

Exemple : si `ls` envoie les deux lignes `fich1.txt` et `fich2.txt` sur sa sortie standard, alors `grep 'e' `ls`` équivaut à `grep 'e' fich1.txt fich2.txt` et affichera les lignes de `fich1.txt` contenant au moins un `e`, suivies des lignes de `fich2.txt` contenant au moins un `e`.

Remarques

- **Différence entre `grep 'e' `ls`` et `ls | grep 'e'` :** la seconde affiche les **noms** de fichiers (parmi ceux retournés par `ls`) qui contiennent au moins un caractère `e` — ce n'est pas la même chose que chercher `e` dans le **contenu** des fichiers.
- Les caractères `<`, `>` ou `>>` peuvent être suivis d'une capture (ex. `grep 'e' < `ls``), mais ce type d'écriture est incorrect si la commande capturée retourne autre chose qu'un nom unique de fichier. **Dans la pratique, on évite ce genre d'écriture.**
- **Différence entre `ls > fich.txt` et `echo `ls` > fich.txt` :** dans les deux cas, le résultat de `ls` est écrit dans `fich.txt`, mais : avec `ls >`, un seul nom de fichier ou de sous-répertoire par ligne ; avec `echo `ls` >`, tous les noms sont écrits sur une **même ligne**, séparés par des espaces (conséquence directe de la transformation des sauts de ligne en espaces lors de la capture).

Redirection de la sortie standard des erreurs d'une commande

- `commande 2> désignation_fichier` (**redirection simple**) : la sortie standard des erreurs de commande est connectée au fichier.
- `commande 2>> désignation_fichier` (**avec concaténation**) : si le fichier n'existe pas, il est créé ; sinon la sortie des erreurs est ajoutée à la fin.
- `commande 2>&1` : la sortie standard des erreurs de commande est connectée à sa **sortie standard**.

```
ls fichier.txt 2> /dev/null
```

écrit dans le fichier « poubelle » `/dev/null` les éventuels messages d'erreur de `ls`. Ce fichier poubelle est un **trou noir** : ce qui y est écrit est définitivement perdu. Utile pour éviter que certains messages d'erreurs ne s'affichent à l'écran.

Les caractères des séquences `2>`, `2>>` ou `2>&1` doivent être **collés**. Ces syntaxes varient d'un shell à l'autre.

Exemples combinés — l'ordre compte !

`ls -l fichier.txt > informations.txt 2>&1` → redirige à la fois la sortie standard **et** la sortie standard des erreurs vers `informations.txt`.

`ls -l fichier.txt 2>&1 > informations.txt` → ne redirige **que** la sortie standard vers `informations.txt`, car la sortie standard des erreurs a été connectée à la sortie standard *avant* que celle-ci ne soit redirigée vers `informations.txt` (les erreurs restent donc à l'écran).

Autre illustration du même piège :

`echo "Problème" 2> erreurs.txt >&2` équivaut à `echo "Problème" > erreurs.txt` → écrit la chaîne `Problème` dans le fichier `erreurs.txt` (stderr est d'abord redirigé vers `erreurs.txt`, puis stdout est redirigé vers ce même stderr, donc vers `erreurs.txt` aussi).

`echo "Problème" >&2 2> erreurs.txt` provoque l'affichage de la chaîne `Problème` à l'écran → car la redirection de stderr vers `erreurs.txt` a été faite *après* la redirection de stdout vers stderr : stdout pointait encore vers l'écran au moment où `>&2` a été exécuté.

Redirections : attention !

Il est **déconseillé** de rediriger l'entrée standard ou la sortie standard **d'un branchement**, pour ne pas écrire des lignes incompréhensibles et pas toujours interprétables par le shell.

```
grep 'e' | grep '2' < fich_entree.txt # AMBIGU : le shell signale une ambiguïté
```

On lui préfère :

```
cat fich_entree.txt | grep 'e' | grep '2'  
grep 'e' fich_entree.txt | grep '2'
```

Remarque : bien que syntaxiquement correcte, la séquence `... | cat | ...` est toujours inutile (un `cat` seul au milieu d'un pipe ne fait que recopier son entrée vers sa sortie, sans rien apporter).

À retenir

- **Trois unités logiques** par processus : `stdin` (clavier), `stdout` (écran), `stderr` (écran) — les redirections changent leurs connexions par défaut.
- **Sortie :** `>` écrase/crée, `>>` ajoute à la fin, `>&2` redirige vers `stderr`. **Entrée :** `<` lit un fichier, `<<séparateur` lit jusqu'au séparateur (utilisable en script, contrairement à `<Ctrl>D`).
- **Pipe** `|` : enchaîne des filtres sans fichier intermédiaire. **Capture** ``...`` (ou `$(...)`) : transforme la sortie d'une commande en chaîne, sauts de ligne remplacés par des espaces.
- **L'ordre des redirections compte :** `2>&1` avant ou après la redirection de `stdout` ne donne pas le même résultat — c'est le piège classique à connaître pour l'examen (voir les deux exemples encadrés en ambre).
- Ne jamais rediriger l'entrée/sortie d'un branchement de commandes (ambigu pour le shell) ; toutes les séquences `2>`, `2>>`, `2>&1`, `>&2` doivent être écrites sans espace.